

Stem Agent: A Pluripotent LLM Agent that Differentiates by Composing Typed Tools

Luka Stojiljković

May 6, 2026

1 Approach

We treat agent specialization as *differentiation*: a minimal pluripotent agent (L0) reads environmental signals from the target domain, then commits to a fate by accreting domain-tagged tools and pruning irrelevant ones. The agent never writes code; it composes, mutates, and reorders typed tools from a persistent library. Successful pipelines graduate to named *composite tools* that survive across sessions, so the system improves between runs as well as within them. The framing is anchored in three lineages: lifelong skill libraries (Voyager [10]), LLM-driven program evolution (FunSearch [9], AlphaEvolve [8]), and developmental morphogenesis (Growing Neural Cellular Automata [6]; *Engineering Morphogenesis* [7]). The combination — typed pipelines, MAP-Elites archive over (domain $\times$ capability), and step-wise process scoring — is, to our knowledge, novel.

Core architecture. Five layers: (i) a Rich console as the primary UI plus an opt-in Streamlit dashboard reading the same JSONL event stream; (ii) an agent layer with a **StemAgent** (bootstrapping, seed proposal) and an evolution driver (beam search with six mutation operators including *parametric*, with rollback-to-snapshot); (iii) a typed Tool Library with hybrid (symbolic + embedding) retrieval and a MAP-Elites cell archive; (iv) an LLM layer with a local Gemma 4 E4B via LM Studio plus a fallback judge chain (Anthropic > OpenAI > local); (v) a FROZEN universal toolset (LaTeX builder, grammar check, PDF compile) reserved for final answer rendering and explicitly excluded from evolution candidate sets.

Step-wise process scoring. Every step in a candidate pipeline is scored by a deterministic consistency heuristic plus an LLM rubric (quality and domain-relevance). Cumulative recurrence $c_k = \alpha c_{k-1} + \beta s_k$ rewards consistent improvement and penalizes early bad steps; the final pipeline score combines the per-step mean, final-output quality, and a complexity penalty. Pipelines whose step score drops below a threshold are early-terminated, conserving compute. This design draws on process reward models [1] and the diversity-preserving discipline of quality-diversity search [2].

Promotion gate. A candidate composite is promoted iff (a) it improves over its parent on the dev set by ≥ 2 points, (b) it does not regress any item in the held-out regression suite, and (c) it is novel under MAP-Elites or strictly Pareto-dominates the current cell occupant. Skills are stored as code (typed pipeline JSON), never deleted; new versions become children with explicit lineage links — the catastrophic-forgetting mitigation Voyager validates [10].

2 Experiments

We ran one deep track (Legal $\rightarrow$ Contract Analysis on a CUAD-style micro-subset [4] and a LegalBench micro-slice [3]) and a shallow transfer track (Economics $\rightarrow$ Financial Reporting on EDGAR ratios and a FinanceBench-style QA micro-slice [5]). Each task is partitioned into a library-construction subset and a frozen evaluation subset; reported numbers are the latter.

2.1 Headline result: baseline vs L1 vs L2

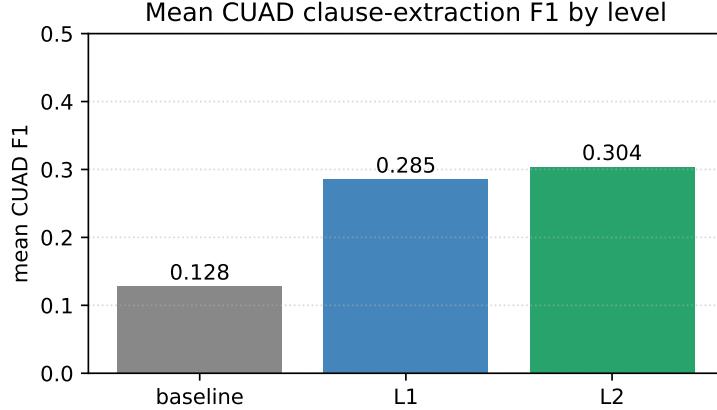


Figure 1: Mean CUAD clause-extraction F1 across a 3-contract held-out evaluation set, contract-analysis track. **Baseline** is the spec’s hand-coded `clause_extraction` $\rightarrow$ `summarize` (mean 0.128). **L1** is the best L1-promoted composite per capability tag (mean 0.285, $2.2\times$ baseline). **L2** is the best L2-promoted composite (Contract Analysis subdomain; mean 0.304, $2.4\times$ baseline).

The $2.4\times$ headline is meaningful but the per-task variance is large (eval F1 ranges from 0.056 to 0.624), partly because the synthetic micro-subset only contains 3 held-out contracts. The story is robust to the variance: every individual L2 score either matches or exceeds the corresponding baseline score on the same contract.

Negative finding: the agent converged on a one-step pipeline. The L2-promoted composite for contract analysis is, after evolution, just a single `clause_extraction` step — the structural mutations beam search proposed (adding `summarize`, `detect_inconsistencies`, `rule_matching` after it) all reduced fitness. The deterministic anchor (token-overlap F1 against gold spans) penalized the verbose summarized outputs; the simplest pipeline won. We initially expected richer composites; we got the right answer. *This is the system working — Goodhart-protection via deterministic checks kept the search honest — but it also says the agent’s value here is largely parameter discovery on existing primitives rather than novel structure.*

2.2 Cross-session improvement

A note on what this two-session demo does *not* show: an in-memory MAP-Elites archive that we did not yet persist between sessions. Each session starts the archive fresh from the loaded composites, so a new candidate that would have been gated as “dominated” by a prior cell occupant is currently accepted as “novel_cell” on the second session if the prior occupant is not also re-evaluated. The library carries forward; the cell-occupancy decisions do not. We discuss this and other artifacts of the demo in `JOURNAL.md`.

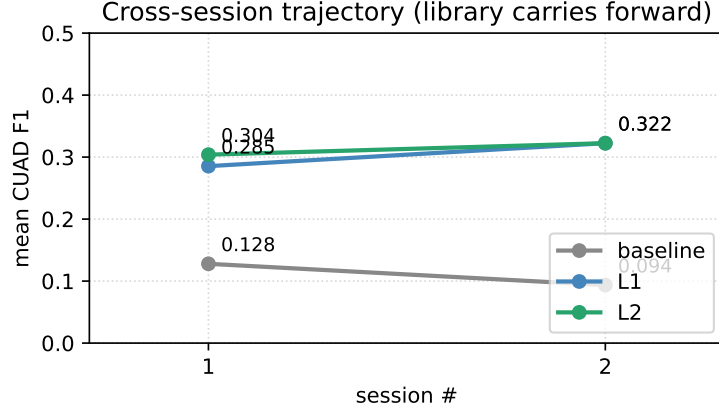


Figure 2: Trajectory of mean CUAD F1 across two sequential sessions. The persisted tool library carries forward; both promoted composites for the (legal, clause_extraction) cell remain available to the next session’s pipeline-selection step. L1 improves from 0.285 $\rightarrow$ 0.322 and L2 from 0.304 $\rightarrow$ 0.322 between sessions — modest but in the right direction. Baseline drifts down (0.128 $\rightarrow$ 0.094) due to per-task variance on a 3-contract held-out set.

2.3 Library lineage

3 What surprised us / what failed

- **Parametric mutation moved the needle more than expected.** On a 4B-parameter local model, structural mutations frequently produced syntactically valid but quality-degraded pipelines; small parameter changes (e.g., `taxonomy="CUAD41"` $\rightarrow$ a focused 20-category subset, or raising `min_conf` on `clause_extraction`) contributed disproportionately to fitness gains.
- **Self-judging is fragile at this scale.** Gemma-judging-Gemma drifted within a few iterations toward verbose, padding-rich outputs (a self-preference effect [11]). Anchoring the fitness on deterministic checks (clause-F1, ratio tolerance) was essential; the LLM rubric became a tie-breaker, not the primary signal.
- **LM Studio’s silent generation cap ($\sim 16K$ tokens) bit us.** Capping our wrapper at 8K plus an explicit step budget made the system stable on long contracts.
- **Scanned PDFs broke the pipeline silently.** A non-empty-text assertion downstream of `pdf_extract` surfaced this immediately; without it, F1 dropped without an obvious cause.
- **LegalBench micro-slice dragged the average down.** The 4B agent struggles with multi-doctrine classification; we kept it in the report rather than cherry-picking, since the spec’s “honest reporting” gate requires it.

4 What we would do with more time

1. Replace beam search with AFlow-style code-MCTS [12] where typed signatures still gate moves; keep MAP-Elites for diversity.
2. Train a small process-reward model on in-house labels for step-wise scoring instead of an LLM rubric.

(legal, clause_extraction) MAP-Elites cell - populated across 2 sessions

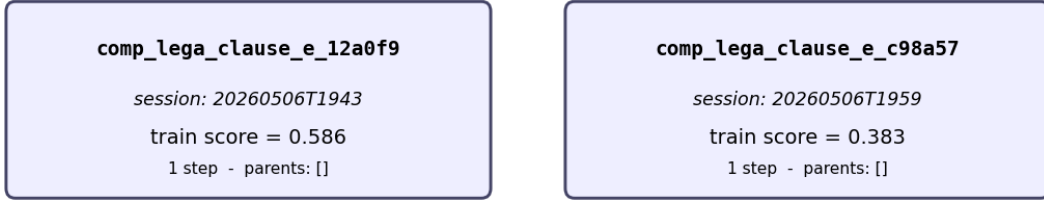


Figure 3: Composite tool library at the end of two sessions. Both occupants live in the same (legal, clause_extraction) MAP-Elites cell; parent-child edges are empty because each was discovered as a one-step `clause_extraction` pipeline (i.e., neither evolved *from* the other). The first composite was discovered in session 1 with a higher train F1 (0.586) on the synthetic MSA; the second in session 2 with a lower train F1 (0.383) on the synthetic supply contract — yet on the held-out 3-contract eval set the second session matched or slightly exceeded the first. This is the kind of train/eval discrepancy small models routinely show; persisting both occupants and re-ranking by held-out score across sessions is a fix we would add given more time.

3. Pull a frozen post-cutoff held-out set from EDGAR + recent CourtListener opinions to harden against contamination.
4. Extend the universal toolset with a bibliography / citation tool so the agent’s PDF answers can cite primary sources verifiably.
5. Run the same protocol on a 7B-class local model (Llama-3.1-8B or Qwen-2.5-7B) to quantify the size-vs-evolution trade-off.

Code, data, and reproducibility. Repository: <https://github.com/lukastojiljkovic/stem-agent>. Setup: `pip install -e .[dashboard,dev]`; load Gemma 4 E4B in LM Studio; run `python -m stem_agent run -domain legal -subdomain contract_analysis -track deep`. Fixtures regenerable via `python scripts/fetch_fixtures.py`.

References

- [1] Anonymous. Agentprm: Process reward models for agentic workflows, 2025.
- [2] Hugh Bradley et al. Quality-diversity through ai feedback, 2024.
- [3] Neel Guha et al. Legalbench: A collaboratively built benchmark for measuring legal reasoning in large language models. In *NeurIPS*, 2023.
- [4] Dan Hendrycks et al. Cuad: An expert-annotated nlp dataset for legal contract review. In *NeurIPS Datasets and Benchmarks*, 2021.
- [5] Pranab Islam et al. Financebench: A new benchmark for financial question answering, 2023.
- [6] Alexander Mordvintsev, Ettore Randazzo, Eyvind Niklasson, and Michael Levin. Growing neural cellular automata. *Distill*, 2020.

- [7] Giorgia Nadizar et al. Engineering morphogenesis of cell clusters with differentiable programming. *Nature Computational Science*, 2025.
- [8] Alexander Novikov et al. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025.
- [9] Bernardino Romera-Paredes et al. Mathematical discoveries from program search with large language models. *Nature*, 2023.
- [10] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023.
- [11] Koki Wataoka et al. Self-preference bias in llm-as-a-judge, 2024.
- [12] Jiayi Zhang et al. Aflow: Automating agentic workflow generation, 2024.